

NLP-Powered Business Intelligence Application

Final Year Project



Asad Kamal
Hamza Akhtar Siddiqui
Talha Yunus

2020090
2020147
2020497

BS Thesis

Faculty of Computer Sciences and Engineering
Ghulam Ishaq Khan Institute of Sciences and Technology

Acknowledgement

We would first like to start off by thanking the one and only Allah Almighty for giving us the perseverance, knowledge and determination to go through and complete this project despite the various hurdles. It also goes without saying that the project wouldn't have seen the light of day without the constant support of our advisor Dr. Taj Muhammad and our sponsor Keystone Consulting, especially Mr. Kheam Khan. We are also very thankful to our friends who were there for us when needed and our families who have supported us throughout our tenure at this Institute and whenever we needed them in our life.

Certificate of Approval

It is certified that the work presented in this report was performed by Asad Kamal, Hamza Akhtar Siddiqui, and Talha Yunus under the supervision of Dr. Taj Muhammad. The work is adequate and lies within the scope of the BS degree in Computer Science at Ghulam Ishaq Khan Institute of Engineering Sciences and Technology.

Dr. Taj Muuhammad
(Advisor)

Dr. Qadeer Ul Hassan
(Dean)

Abstract

This project aims to simplify business intelligence reporting that includes KPIs, dashboards, reports, and other insightful analyses by creating an application that integrates multiple databases, fetches the required data, and visualizes the information through graphs, reports, dashboards, etc. The web application will also include an NLP-based chatbot allowing the end-user to converse with the application and obtain the desired reports/charts without requiring technical knowledge hence the project will help consumers, who are dependent on IT users, and who do not possess the technical background and skillset to generate these BI artifacts. Advanced natural language understanding techniques and open-source large language models have been implemented for the chatbot; additionally, the application includes a user-friendly frontend that allows end-users to manage multiple databases, dashboards, and their user profile and other settings.

Table of Context

Acknowledgement	2
Certificate of Approval	3
Abstract	4
Chapter 1	9
1. Introduction	9
1.1. Motivation	9
1.2. Preface	9
1.3. Overall Description	9
1.4. Product Perspective	10
Chapter 2	11
2. Literature Survey	11
2.1. Defog SQLCoder	11
2.2. Llama2	12
2.3. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models	13
2.4. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL	14
2.5. Graphix-T5: Mixing Pre-Trained Transformers with Graph-Aware Layers for Text-to-SQL Parsing	14
2.6. Tableau	15
2.7. PowerBI	17
	5

Chapter 3	19
3. Design (Systems Requirements /Specifications)	19
3.1 Intro	19
3.2 Product Perspective	19
3.3 Product Functions	19
3.4 User Characteristics	20
3.5 Constraints	20
3.6 Assumptions and Dependencies	21
3.7 External Interface Requirements	21
3.8 Functional Requirements	23
3.9 Non Functional Requirements & Software System Attributes	26
3.9.1 Response Time	26
3.9.2 Scalability	27
3.9.3 Concurrency	27
3.9.4 Database Performance	27
3.9.5 Logging and Monitoring	27
3.9.6 Load Testing	27
Chapter 4	28
4. Proposed Solution	28
4.1. Methodology	28
4.1.1. Work Breakdown Structure	28
4.1.2. Architectural Design	29
	6

4.1.3. Why do we choose Layered Architecture Design.....	30
4.2 4+1 Architecture View Model	31
4.2.1 Logical View	31
4.2.2 Development View	32
4.2.3 Process View	33
4.2.4 Physical View	34
4.2.5 Use Case	35
4.3 Implementation	36
Chapter 5	43
5. Results and discussion	43
Chapter 6	45
6. Conclusion and future work	45
References	46
Appendix A	47

List of Figures

Figure 4.1	28
Figure 4.2	29
Figure 4.3	31
Figure 4.4	32
Figure 4.5	33
Figure 4.6	34
Figure 4.7	35

List of Tables

Table 1	24
Table 2	25
Table 3	25
Table 4	26

Chapter 1

1. Introduction

1.1. Motivation

Businesses today drown in a sea of data, each drop holding the potential for valuable insights. Yet, traditional Business Intelligence (BI) tools are often complex, requiring specialized technical knowledge. This creates a bottleneck, hindering access to crucial information and stifling business agility.

1.2. Preface

This project tackles this challenge by introducing a revolutionary BI application designed specifically for non-technical users. It's a paradigm shift, empowering everyday employees to become data heroes and fostering a culture of data-driven decision making across all departments and roles.

1.3. Overall Description

The cornerstone of this initiative is user empowerment. We've meticulously crafted a user-friendly interface that eliminates technical jargon and provides seamless access to the entire suite of BI tools. It acts as a key, unlocking a treasure chest overflowing with valuable insights.

Furthermore, the application boasts robust data integration capabilities, seamlessly connecting with various backend databases and acting as a bridge to gather data from disparate sources for a holistic

view. This is particularly crucial in today's interconnected business landscape, allowing users to harness the power of data wherever it resides and break down information silos.

1.4. Product Perspective

This is just the beginning. We envision exciting possibilities for commercial expansion through integration with popular third-party applications via APIs. Imagine gaining a holistic view of marketing campaigns, sales pipelines, and customer behavior – all presented in a user-friendly format. Our groundbreaking BI application isn't just about data; it's about people. We're building a future where everyone has the power to unlock the secrets hidden within the data deluge.

Our project empowers users through data visualization tools and the potential of natural language processing in business intelligence.

Chapter 2

2. Literature Survey

2.1. Defog SQLCoder

Defog SQLCoder is a new large language model (LLM) designed to translate natural language questions into SQL queries. This technology bridges the gap between human communication and database interaction.

Natural Language to SQL is a complex task because natural language is unstructured while SQL queries are very specific. Researchers have explored various techniques to tackle this challenge, such as semantic parsing (converting natural language into a format suitable for query generation) and code generation with LLMs (using large language models to write code directly from descriptions).

Defog SQLCoder outperforms existing open-source models and even rivals larger models like GPT-3.5-turbo on specific benchmarks. This suggests it's effective at translating natural language questions into SQL queries. The model is a 15 billion parameter LLM built upon a fine-tuned version of StarCoder.

Defog SQLCoder utilizes a hand-crafted training dataset with varying difficulty levels and diverse schemas, indicating a focus on robust training and adaptability. The document describes a two-stage

fine-tuning process, suggesting a targeted approach to improve performance on complex queries.

Future research directions include comparing Defog SQLCoder with other advanced LLMs, understanding how it generates SQL queries, and exploring its ability to adapt to entirely new database schemas. Overall, Defog SQLCoder shows promise for the field of NL2SQL generation due to its reported performance and focus on high-quality training data.

2.2. Llama2

NSQL-Llama-2-7B is a new large language model designed specifically for converting natural language descriptions into SQL queries. This literature survey explores how NSQL-Llama-2-7B fits into the existing research on LLMs for SQL generation.

Traditionally, converting natural language into SQL queries has been challenging due to the differences in structure and meaning between the two. Researchers have explored various techniques to address this, such as semantic parsing (converting natural language to a format suitable for queries) and using encoder-decoder models (LLMs that can directly generate code from descriptions).

NSQL-Llama-2-7B builds upon existing work by leveraging Meta's open-source Llama-2 LLM. This suggests that NSQL-Llama-2-7B benefits from the transparency and strong performance of Llama-2.

The training strategy for NSQL-Llama-2-7B involves two stages: pre-training on a general dataset of SQL queries and then fine-tuning

on specific text-to-SQL pairs. This aligns with research on transfer learning for LLMs in code generation tasks.

NSQL-Llama-2-7B evaluated using established benchmarks like Spider and GeoQuery. This allows for comparison with other text-to-SQL models. However, it's important to note that NSQL-Llama-2-7B is currently limited to generating specific types of SQL queries (SELECT statements) based on particular prompt formats.

Overall, NSQL-Llama-2-7B shows promise for text-to-SQL generation by leveraging the capabilities of Meta's Llama-2 LLM. Further research could solidify its position in this field. This includes comparing it with other LLMs, exploring its potential for more complex queries, and making it more interpretable for debugging and user trust.

2.3. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models

The paper showed that PICARD uses a parsing library called attoparsec that supports incremental input. This is a special capability that is not available in many other parsing libraries. You can feed attoparsec a string that represents only part of the expected input to parse. When parsing reaches the end of an input fragment, attoparsec will return a continuation function that can be used to continue parsing. Input fragments can be parsed one after the other when they become available until the input is complete. This means that it takes chunks of the input and parses them in that way.

2.4. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL

In RESDSQL, a seq2seq model architecture is employed to facilitate the schema linking and skeleton parsing processes. This architecture comprises both an encoder and a decoder. The encoder's role is to extract contextual information from the input natural language, encompassing the question and the database schema. It consists of a sequence encoder and a graph encoder. Subsequently, the decoder utilizes this contextual information to generate SQL queries while also performing skeleton parsing. To ensure the correctness and coherence of the generated SQL queries, specialized grammar- and execution-guided decoders are utilized. Notably, RESDSQL initiates the schema linking process by ranking schema links based on the input natural language, enabling the selection of pertinent tables and columns from a potentially extensive and intricate schema, thereby enhancing query relevance to the question.

2.5. Graphix-T5: Mixing Pre-Trained Transformers with Graph-Aware Layers for Text-to-SQL Parsing

This paper showed the model Graphix-T5 which is a text-to-SQL parsing system that harnesses the combined power of pre-trained transformers and graph-aware layers to effectively bridge natural language understanding and SQL query generation. It leverages two key pre-trained transformers: T5, which encodes the natural language question and generates the SQL query, and a novel graph-aware transformer, specially designed for reasoning over database structures. This graph-aware transformer is trained on top of T5 and then

fine-tuned on text-to-SQL queries, making it highly adaptable to unseen databases. In the Graphix-T5 workflow, the natural language question is initially tokenized, followed by encoding with the pre-trained transformer encoder. Subsequently, the encoded hidden states are passed to the graph-aware decoder, responsible for SQL query generation by employing graph representation reasoning. Finally, the generated SQL query is executed against the database, yielding the desired results. This approach seamlessly combines the strengths of pre-trained models and graph-aware processing for efficient text-to-SQL parsing.

2.6. Tableau

Tableau has carved a niche as a leading data visualization software, renowned for its ability to transform raw data into clear and impactful insights. Its popularity stems from its user-friendly approach, streamlining the data analysis process. Unlike traditional methods that require complex coding, Tableau empowers users to establish connections between tables directly from databases. This intuitive drag-and-drop interface fosters a visual exploration of data, allowing users to effortlessly build informative visualizations by simply dragging and dropping relevant data attributes onto designated sheets. This ease of use makes Tableau a powerful tool for a wide range of users, from seasoned data analysts to individuals with a basic understanding of data.

However, it's important to acknowledge that unlocking the full potential of Tableau requires a foundational understanding of data

concepts. While the drag-and-drop interface offers an inviting entry point, it assumes a baseline knowledge of data structures, terminology, and underlying principles. For instance, effectively navigating table relationships within Tableau hinges on a grasp of relational database concepts. Users need to understand how tables are connected, how data is organized within them, and how to leverage these relationships to extract meaningful insights. Additionally, proficiency in SQL, a database query language, can significantly enhance the Tableau experience. SQL allows users to tailor data retrieval from databases, ensuring they retrieve only the specific data segments relevant to their analysis. This targeted data retrieval not only streamlines the visualization process but also optimizes Tableau's performance.

Tableau's drag-and-drop interface might create the illusion of a completely accessible tool, allowing users to translate complex data structures and relationships into visually compelling stories. However, this translation process necessitates a foundational understanding of the data itself. Hence, while Tableau offers a user-friendly interface, it assumes a certain level of technical fluency in data concepts and, ideally, proficiency in SQL. This can pose a challenge for non-technical users who might lack prior experience with data analysis or database querying.

In conclusion, Tableau is a powerful tool for data visualization, offering an intuitive and user-friendly interface. However, to truly harness its potential and unlock the hidden stories within data, users need to invest some time in learning basic data concepts and, ideally, develop proficiency in SQL. By acknowledging this learning curve, users can

approach Tableau with realistic expectations and leverage its capabilities to transform data into impactful insights, ultimately leading to more informed decision-making.

2.7. PowerBI

Power BI has emerged as a dominant force in the area of business intelligence (BI) tools. Renowned for its versatility and power, it empowers users to embark on a comprehensive data exploration journey. At its core, Power BI facilitates seamless connection to a vast array of data sources, encompassing everything from traditional databases to cloud-based repositories. This eliminates the tedious task of data migration, allowing users to focus on extracting valuable insights from the outset.

Beyond data acquisition, Power BI offers robust data transformation and modeling capabilities. Raw data, often messy and unrefined, can be meticulously sculpted into a structured and organized format. This transformation process equips the data for insightful analysis. Power BI provides a user-friendly interface for data cleansing, filtering, and manipulation, ensuring the data reflects the specific needs of the analysis. Additionally, data modeling techniques allow users to establish meaningful relationships between disparate data sets, fostering a holistic understanding of the information landscape.

Once the data is meticulously prepared, Power BI empowers users to craft interactive reports and dashboards. These reports and dashboards are not static presentations; they are dynamic and engaging experiences. Users can leverage a plethora of visual elements, from

charts and graphs to maps and tables, to present complex data in a clear and consumable way. Furthermore, the interactive nature of these reports allows users to drill down into specific data points, filter information based on their needs, and uncover hidden trends or patterns. This level of interactivity fosters deeper exploration and a more comprehensive understanding of the data.

However, the very features that make Power BI so powerful can also present a significant hurdle. Unlocking the full potential of Power BI necessitates a foundational understanding of data concepts. Users need to possess a working knowledge of data structures, data types, and data governance principles. While the user interface is intuitive, effectively navigating data transformation and modeling tasks requires a level of technical fluency. Furthermore, proficiency in SQL, a database query language, can significantly enhance the Power BI experience. SQL allows users to precisely extract the specific data segments relevant to their analysis, optimizing the data preparation process and maximizing efficiency.

In conclusion, Power BI stands as a powerful BI tool, offering a comprehensive suite of features for data exploration, transformation, and visualization. However, its true potential can only be fully realized by users equipped with a foundational understanding of data concepts and, ideally, proficiency in SQL. By acknowledging this learning curve, potential users can approach Power BI with realistic expectations. For those willing to invest the time and effort in acquiring the necessary technical skills, Power BI can be an invaluable

asset, transforming raw data into actionable insights that drive informed decision-making.

Chapter 3

3. Design (Systems Requirements /Specifications)

3.1 Intro

The project aims to address the challenges faced by businesses in generating meaningful business intelligence (BI) reports and dashboards. It proposes the development of an application that integrates with multiple backend databases, retrieves necessary data, and presents it in the form of graphs, reports, and dashboards. Additionally, a natural language processing (NLP) based chatbot will be implemented to allow non-technical business users to interact with the application and request specific reports without requiring technical expertise. The project will focus on using open-source databases like PostgreSQL and MySQL for proof of concept, along with popular web frontends and charting libraries.

3.2 Product Perspective

The proposed project introduces a novel perspective within the domain of business intelligence tools and applications. It aims to bridge the gap between technical complexities and the needs of non-technical business users.

3.3 Product Functions

The key functions of the proposed application are as follows:

- Integration with multiple backend databases to fetch data.
- Visualization of data through various means such as graphs, reports, and dashboards.
- Implementation of an NLP-based chatbot for user interaction.
- Capability to generate BI reports based on user queries.
- Compatibility with open-source databases like PostgreSQL and MySQL.

3.4 User Characteristics

The primary users of this application are business professionals who require BI insights but may not possess technical expertise in generating BI assets. These users include sales managers, HR managers, and other non-technical personnel within organizations. Additionally, IT users responsible for maintaining the application and databases are also considered users.

3.5 Constraints

The project operates within the following constraints:

- Proof of concept limited to one or two open-source databases (e.g. PostgreSQL and MySQL).
- Development of accurate NLP chatbot due to the incompetency of currently available LLM models.

- Availability of GPUs with the required VRAM as LLM models require a large amount of VRAM to train and fine-tune.

3.6 Assumptions and Dependencies

The project assumes the following:

- Availability of open-source databases for integration.
- Access to relevant data within the chosen databases.
- Adequate hardware for application development and deployment.
- Potential expansion to include integration with other applications through APIs based on project success.

3.7 External Interface Requirements

3.7.1 User Interfaces

This interface will be designed for non-technical business users who need access to business intelligence reports and insights. It will feature a user-friendly, intuitive dashboard where users can interact with the NLP-based chatbot. Users will be able to communicate their data needs through natural language queries.

3.7.2 Hardware Interfaces

This application does not require any specific hardware requirements. The web application will be able to run standard computing hardware that can run a web browser. Additionally, an

internet connection will be required by the user to access the application and run it.

3.7.3 Software Interfaces

This application will integrate various software components and libraries:

The frontend framework will be built on React.js, an open source JavaScript library. The frontend will also utilize various JavaScript libraries and packages such as Chart.js for data visualization.

The backend framework will be built using Flask, a Python web framework. The primary functions of the backend include handling requests from the frontend, facilitating communication with the LLM responsible for SQL query generation, and servicing XMTP requests to interact with the database.

This application will also interface with backend databases such as MySQL and PostgreSQL; this interface will enable the retrieval of data to generate data visualizations and insightful reports.

Another key component of this project will be the Large Language Model that will be able to generate SQL queries from natural language prompts. This component forms a crucial part of the software's user interface, enabling users to interact with the application using natural language queries.

3.7.4 Communication Interfaces

This application will employ various communication interfaces to ensure seamless flow of data and interactions between different components of the project.

HTTP Communication: The frontend of the application will communicate with the backend using HTTP to handle exchange of data and requests between the user interface and the server-side components.

XMTP Requests: This application will also use XMTP requests to interact with the backend databases. These requests will be used for data retrieval from backend databases such as MySQL and PostgreSQL.

REST APIs: The LLM component of the application will be accessed through Restful API call, allowing data communication with the user and the LLM. Firebase's Firestore API will also be used to allow communication with the cloud database of the application, enabling real-time data storage and synchronization.

3.8 Functional Requirements

FR-1. SQL Generation via NLP:

- a. Implement Natural Language Processing (NLP) for SQL query generation.
- b. Interpret user queries in natural language and convert them into SQL queries.

FR-2. SQL Verification for correctness:

- c. Validate generated SQL queries for correctness.
- d. Establish acceptance criteria for SQL query accuracy.

FR-3. Flexible data visualization:

- e. Offer a diverse range of data visualization options.
- f. Allow users to customize and edit visualizations as needed.

FR-4. Integration with Multiple databases:

- g. Integrate at least two types of databases for testing.
- h. Include both open-source and off-the-shelf databases.
- i. Ensure efficient data retrieval from integrated databases.

3.8.1 Functional Requirements with Traceability Information

Requirement ID	FR-1		Requirement Type	Functional		Use Case #		UC-1	
Status	<i>New</i>	-	<i>Agreed-to</i>	X	<i>Baselined</i>	-	<i>Rejected</i>	-	
Parent Requirement #	None								
Description	Implement Natural Language Processing (NLP) for SQL query generation. Interpret user queries in natural language and convert them into SQL queries.								
Rationale	This functionality enhances user experience by allowing them to query databases using natural language, making it more accessible and user-friendly.								
Source	Business Stakeholders				Source Document		-		
Acceptance/Fit Criteria	Successful NLP conversion of user queries into valid SQL queries with a minimum accuracy rate of 75%.								
Dependencies	None								
Priority	<i>Essential</i>		X	<i>Conditional</i>		-	<i>Optional</i>	-	
Change History									

Table 1

Table 1: Functional Requirements FR-1

Requirement ID	FR-2		Requirement Type	Functional		Use Case #		UC-2	
Status	New	-	Agreed-to	X	Baselined	-	Rejected	-	
Parent Requirement #	None								
Description	Validate generated SQL queries for correctness. Establish acceptance criteria for SQL query accuracy.								
Rationale	Ensures that the SQL queries generated from natural language queries are accurate and reliable for data retrieval.								
Source	Database Administrators				Source Document		-		
Acceptance/Fit Criteria	SQL query correctness verified through automated testing with a minimum success rate of 80%.								
Dependencies	FR-1 (SQL Generation via NLP)								
Priority	Essential		-	Conditional		-	Optional	X	
Change History									

Table 2

Table 2: Functional Requirements FR-2

Requirement ID	FR-3		Requirement Type	Functional		Use Case #		UC-3	
Status	New	-	Agreed-to	X	Baselined	-	Rejected	-	
Parent Requirement #	None								
Description	Offer a diverse range of data visualization options. Allow users to customize and edit visualizations as needed.								
Rationale	Enhances the usability of the system by enabling users to analyze and present data in various ways, tailored to their specific needs.								
Source	Data Analysts				Source Document	-			
Acceptance/Fit Criteria	Provide at least five different data visualization types (e.g., bar charts, pie charts, line graphs), and allow users to customize colors, labels, and other visual properties.								
Dependencies	FR-1 (SQL Generation via NLP)								
Priority	Essential		X	Conditional	-	Optional	-		
Change History									

Table 3

Table 3: Functional Requirements FR-3

Table 4: Functional Requirements FR-4

Requirement ID	FR-4		Requirement Type		Functional		Use Case #		UC-4
Status	Ne w	-	Agreed-to	X	Baselined	-	Rejected	-	
Parent Requirement #	None								
Description	Integrate at least two types of databases for testing. Include both open-source and off-the-shelf databases. Ensure efficient data retrieval from integrated databases.								
Rationale	Provides users with flexibility by allowing them to connect to various database systems and retrieve data efficiently.								
Source	Database Administrators				Source Document		-		
Acceptance/Fit Criteria	Successful integration with at least two databases, one open-source and one off-the-shelf, and demonstrate efficient data retrieval within acceptable time limits.								
Dependencies	None								
Priority	Essential		X	Conditional		-	Optional	-	
Change History									

Table 4

3.9 Non Functional Requirements & Software System Attributes

3.9.1 Response Time

The BI application should deliver responsive performance. Queries, report generation, and data visualization operations should occur within a reasonable timeframe to ensure that users receive timely insights. The response time for generating SQL queries and rendering visualizations should be optimized for a seamless user experience.

3.9.2 Scalability

The application should be designed to handle varying workloads and data volumes. It must scale efficiently to accommodate increasing numbers of users and data sources without significant degradation in performance. Scalability should be achieved through load balancing, efficient resource allocation, and data optimization techniques.

3.9.3 Concurrency

The application should support concurrent user interactions. It must handle multiple users executing queries and generating reports simultaneously without system slowdowns or conflicts. Effective concurrency management is crucial to maintain responsiveness.

3.9.4 Database Performance

Performance considerations should extend to the integrated databases. The application should optimize database queries and minimize database response times to ensure that data retrieval is swift and efficient.

3.9.5 Logging and Monitoring

Implement comprehensive logging and monitoring capabilities to track application performance, detect anomalies, and allow for proactive performance optimization. Log data should be accessible for analysis.

3.9.6 Load Testing

Conduct rigorous load testing under varying conditions to assess and validate performance requirements. This should include simulating heavy workloads and user concurrency to identify potential bottlenecks and fine-tune performance.

Chapter 4

4. Proposed Solution

4.1. Methodology

4.1.1. Work Breakdown Structure

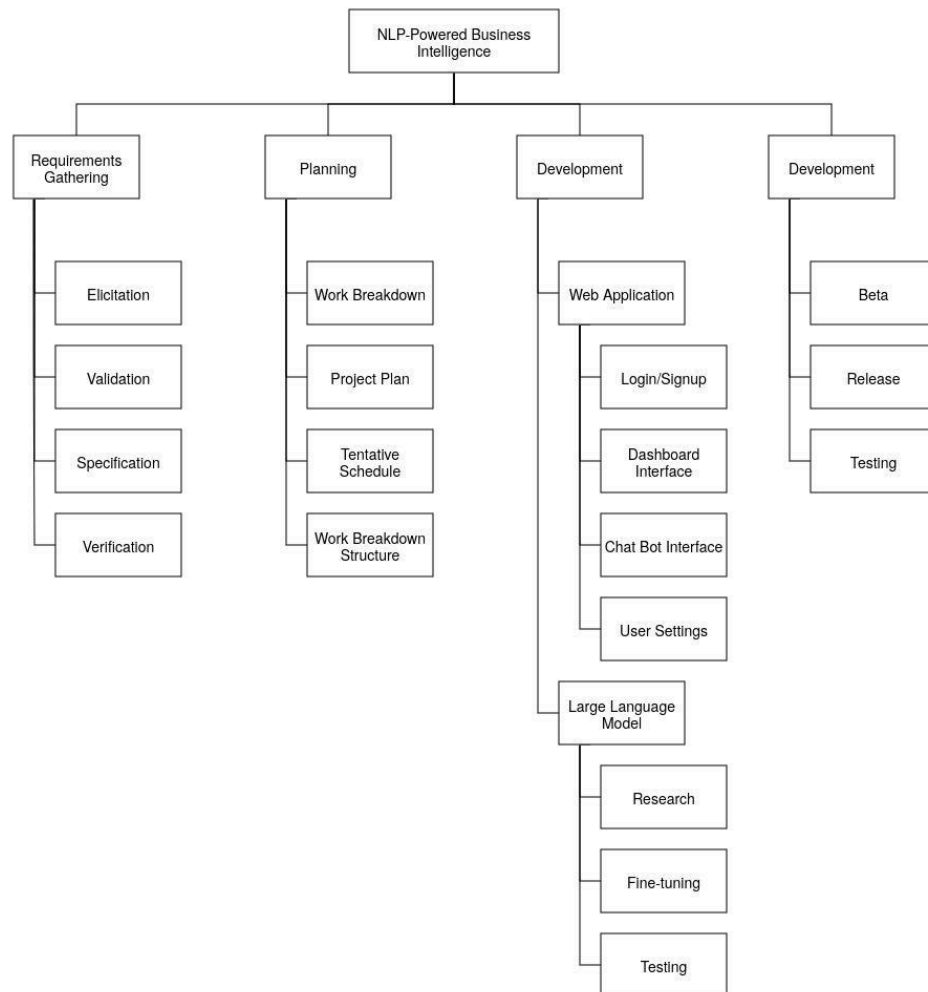


Figure 4.1

4.1.2. Architectural Design

The application uses the layered architectural pattern.

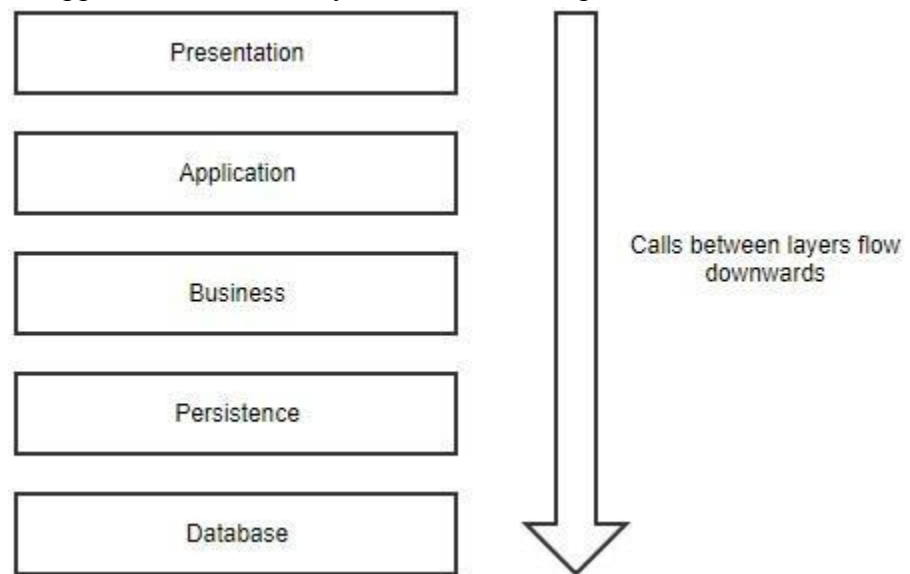


Figure 4.2

Imagine the inner workings of our application as a layered cake. The top layer, the presentation layer, is all about how users interact with the application. This is where they see information and give instructions. Next comes the application layer, which acts like the cake's filling. Here, the code we've written translates user interactions into actions. Below that sits the business layer, the foundation of the cake. This layer handles the core logic of the problem we're trying to

solve, ensuring the application functions as intended. Finally, at the very base lies the persistence layer, which acts like the cake plate. It provides a hidden layer of support for the business layer and interacts with the database, where all the data is stored. Each layer works together seamlessly to deliver a smooth user experience and tackle the business challenge at hand.

4.1.3. Why do we choose Layered Architecture Design?

Layered architecture is a popular design choice for several reasons. It promotes flexibility and maintainability by separating the user interface, business logic, and data access into distinct layers. This separation of concerns allows for clean and independent development of each layer. Imagine a layered cake - the user interface is the frosting, the business logic is the cake itself, and the data access layer is the plate. You can easily swap out the frosting (UI) for a different flavor (web interface vs mobile app) without affecting the rest of the cake. Additionally, layered architecture promotes reusability. Components like business logic can be reused across different applications, saving development time and effort. Furthermore, layered architecture fosters parallel development. Different teams can work on separate layers simultaneously, minimizing dependencies and speeding up the development process. Finally, layered architecture enables loosely coupled systems. Each layer can be independently deployed, maintained, and updated, offering greater control over security and scalability. You can choose to secure sensitive logic behind a firewall while keeping the user

interface accessible from anywhere. Layered architecture also simplifies testing by allowing for isolated unit testing of each layer.

4.2 4+1 Architecture View Model

4.2.1 Logical View

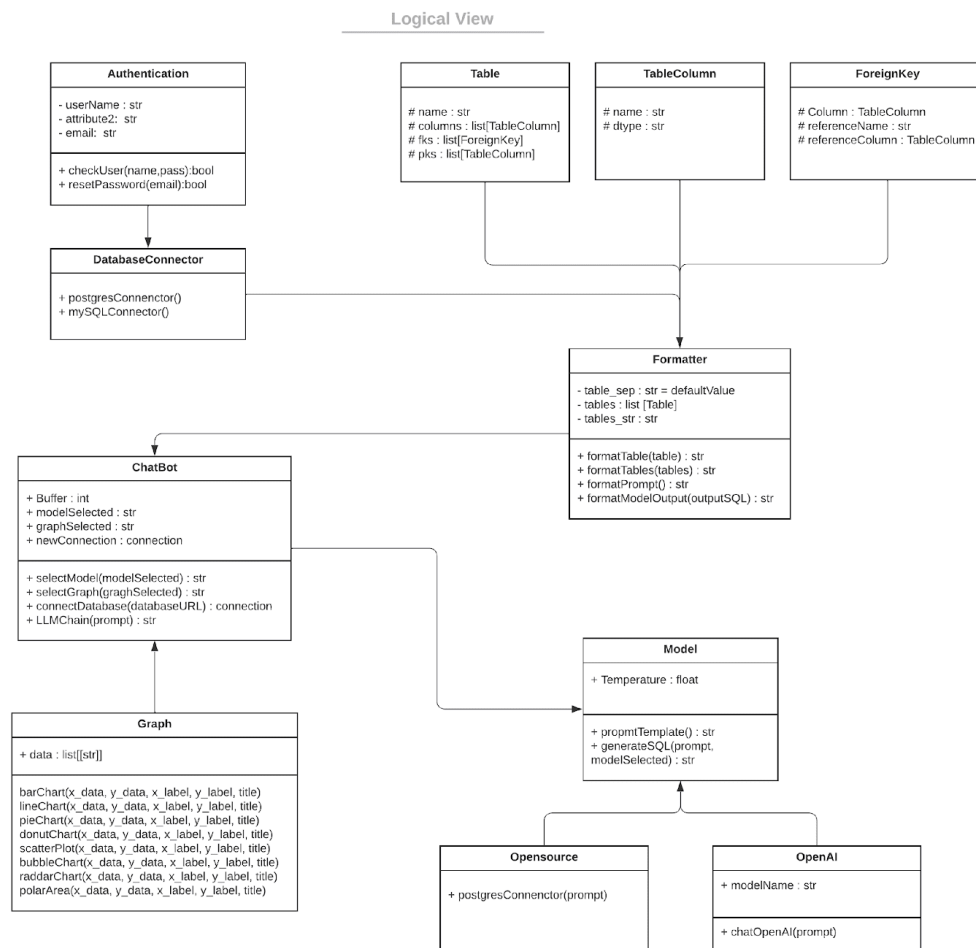


Figure 4.3

4.2.2 Development View

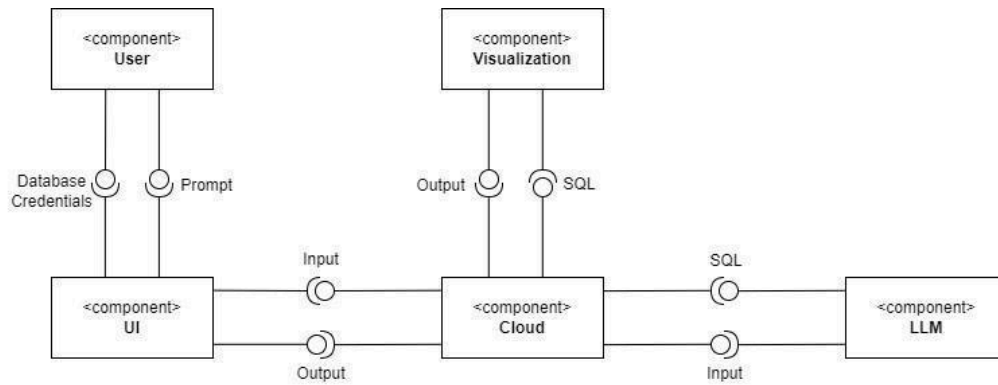


Figure 4.4

4.2.3 Process View

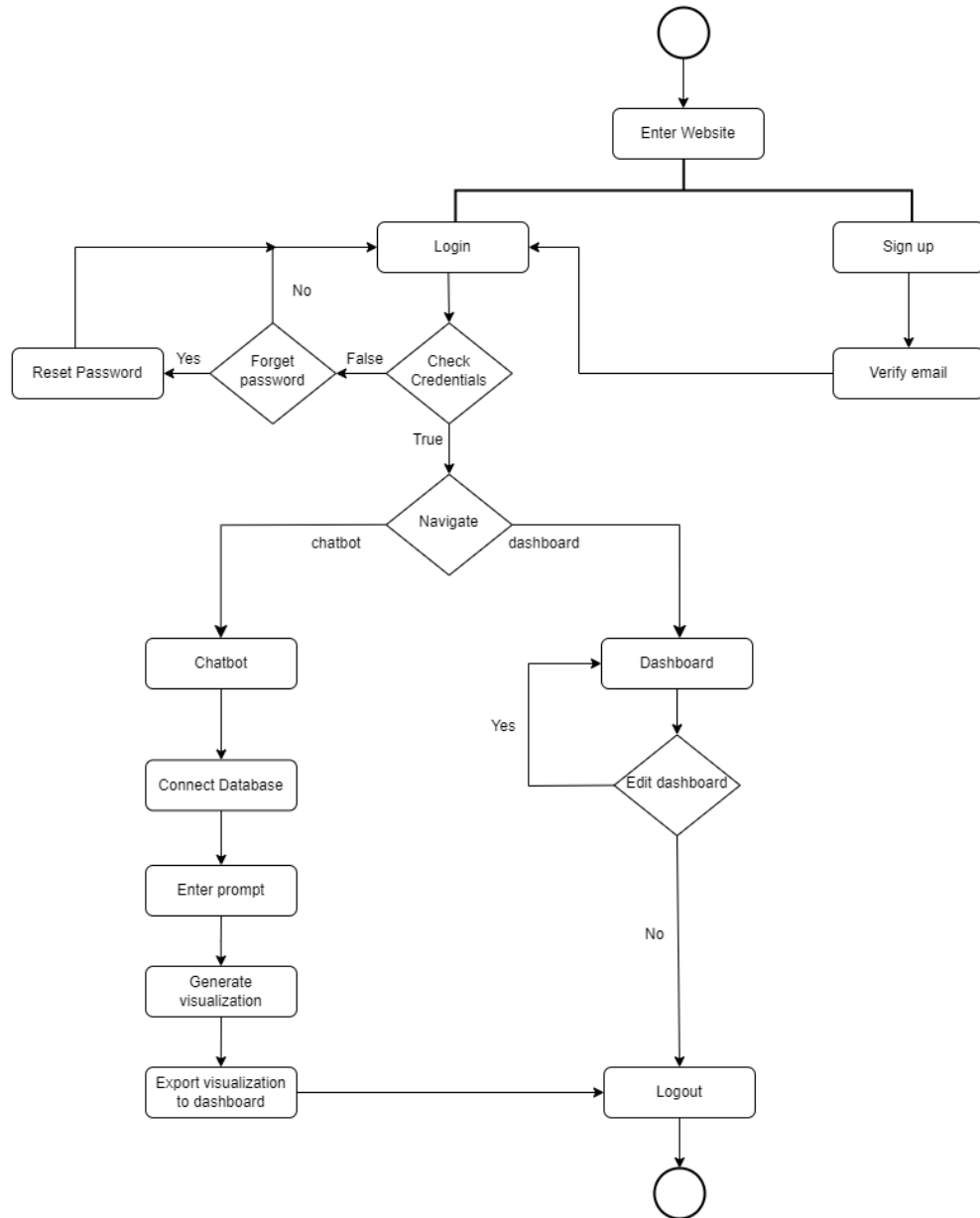


Figure 4.5

4.2.4 Physical View

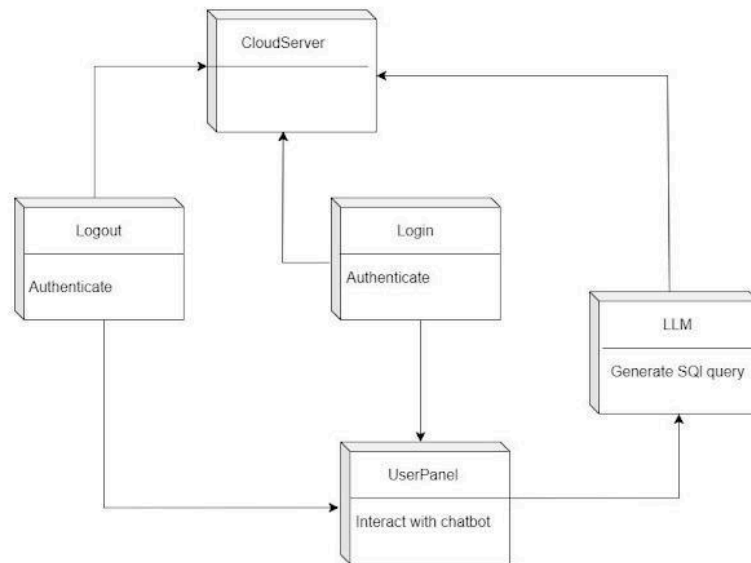


Figure 4.6

4.2.5 Use Case

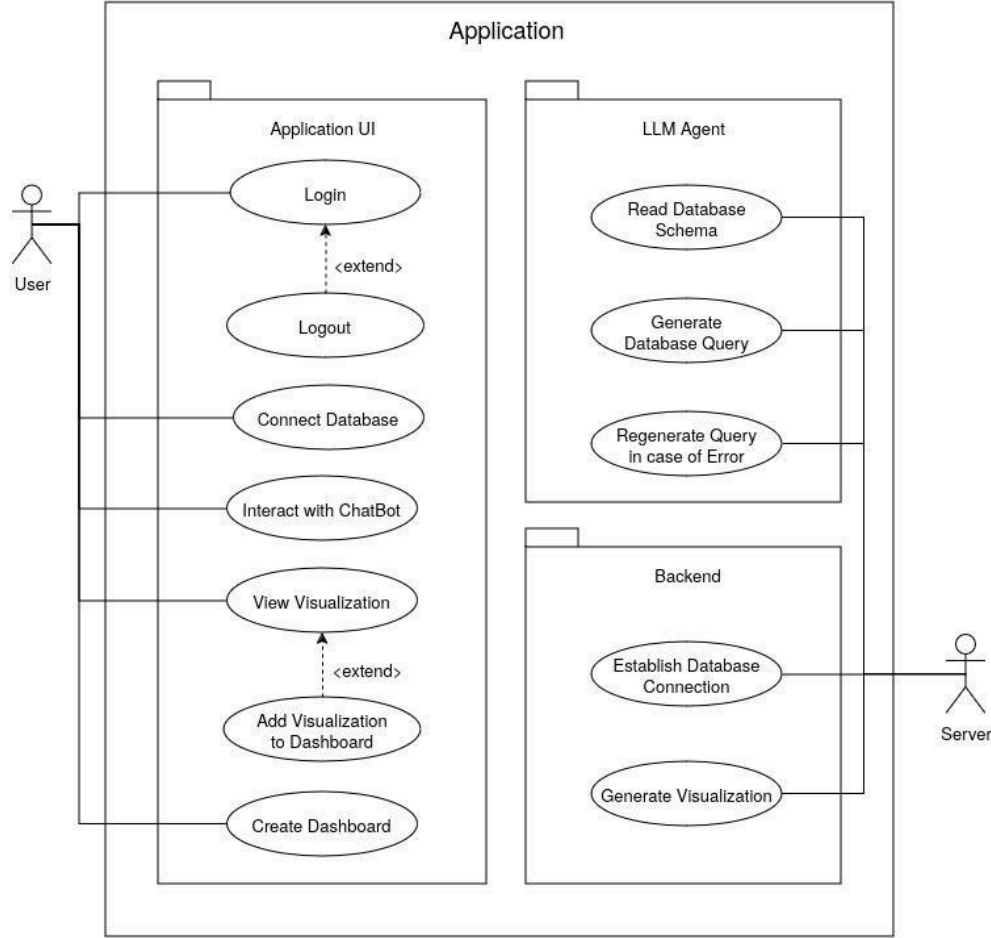


Figure 4.7

4.3 Implementation

4.3.1. User registration

Allows secure login and personalization, the system allows user registration via email and password. Email verification ensures account validity, and login utilizes secure credential checks with session management. Furthermore, users can reset their password in case they forget their passwords.

- **Sign in and Signup:** To ensure a secure and personalized user experience, the system offers a robust user registration process. Users can create accounts by registering with their email address, which doubles as their login credential. A secure password meeting complexity requirement is also chosen during registration. Users will have to provide their first and last names for personalization. All user information, especially passwords, is securely stored using industry-standard hashing algorithms, preventing plain text storage and mitigating security breaches. To implement the user sign in/signup we used firebase in our backend to store the user credentials.
- **User Authentication:** The system utilizes a secure login process to verify user identities. A readily accessible login form allows users to enter their registered email and password. This information is then checked against the stored credentials in the database. We used Firestore to employ a secure hashing algorithm for password verification, ensuring only users with the correct credentials can

access the system. Successful login establishes a secure session for the user, enabling them to interact with the application. This session might involve generating a token for authorization purposes.

- **Password Reset:** To ensure users can regain access even if they forget their password, the system offers a flexible password reset mechanism. Users can initiate the reset process through their registered email address, receiving a password reset link via Firestore for easy password updates.
- **Session Management:** Session management plays a crucial role in maintaining system security and user experience. Sessions are configured to expire after a period of inactivity, automatically logging out idle users. This helps prevent unauthorized access if a user leaves their device unattended. In essence, session management ensures that only authenticated users have access to the system and that sessions don't remain active indefinitely, promoting overall system security.

4.3.2. Database Management

The chatbot empowers users to manage their data access by enabling them to add new databases as sources and remove unwanted ones. The user's databases are stored by saving their URL to the firebase database. This functionality provides flexibility and control over the data used within the chatbot interactions.

- **Add Database:** The chatbot empowers users to expand their data sources through a user-friendly "Add Database" functionality. This

allows users to connect the chatbot to new databases, enriching the potential insights gleaned from user interactions. The process likely involves providing the chatbot with relevant connection details specific to the database, enabling it to seamlessly access the data for analysis and visualization and save the connection details so that it can be accessed again later without needing to provide the connection details again.

- **Remove Database:** The chatbot recognizes the need for data management flexibility. The "Remove Database" functionality allows users to declutter their data sources by eliminating databases that are no longer relevant or needed. By doing this the database connection details are also removed from our firebase database. This ensures the chatbot operates on the most current and pertinent data sets, fostering more accurate and focused results. Removing a database involves a confirmation step to prevent accidental deletions.
- **Manage Multiple Databases:** The chatbot acknowledges that users might require data from various sources. To address this, it offers "Manage Multiple Databases" functionality. This empowers users to connect and manage a collection of databases within the chatbot interface. Users can easily switch between these databases depending on their specific needs, allowing for a comprehensive exploration of data from diverse sources. This functionality streamlines the data exploration process and empowers users to leverage a wider range of information within their chatbot interactions.

4.3.3. Dashboard Managment

The system facilitates user control over data visualization by allowing them to create and manage multiple dashboards. Users can add dashboards to their account to revisit saved visualizations, promoting efficient exploration and analysis of their data. The dashboards are created using ReactJS and the SQL query of charts in every dashboard are saved in the firebase database.

- **Add Dashboard:** The system empowers users to tailor their data exploration experience through the "Add Dashboard" functionality. Users can create new dashboards within their account, allowing them to organize and group specific visualizations for easy access and reference. This functionality likely involves selecting the desired visualizations and assigning a meaningful name to the dashboard for future identification. Adding dashboards promotes user control over data exploration and fosters a personalized visualization experience.
- **Remove Dashboard:** The system recognizes that user needs may evolve, and it provides the ability to remove unwanted dashboards through the "Remove Dashboard" functionality. This allows users to declutter their workspace and eliminate dashboards that are no longer relevant or useful. A confirmation step during removal helps prevent accidental deletion of important dashboards.
- **Manage Multiple Dashboards:** The system acknowledges the complexity of data analysis, often requiring exploration from multiple perspectives. To address this, it offers "Manage Multiple Dashboards" functionality. Users can create and manage a collection

of dashboards within their account. This allows users to switch between different dashboards depending on their current focus, facilitating a comprehensive understanding of their data. Managing multiple dashboards empowers users to personalize their data exploration journey and efficiently revisit saved visualizations for further analysis.

- **View Saved Visualizations:** The core function of dashboards lies in their ability to store and display saved visualizations. Users can access their personalized collection of dashboards at any time, allowing them to revisit previously created visualizations. This functionality promotes efficient data exploration and analysis by eliminating the need to recreate visualizations repeatedly. By readily viewing saved visualizations, users can gain deeper insights from their data and make informed decisions.

4.3.4. NLP Agent

This NLP agent acts as a bridge between users and their databases. Users can connect, chat-style, to get data visualizations based on their prompts. They even have control over the visualization format, fostering an interactive data exploration experience. The user connects to the database, the backend gets the schema of that database and stores it in a vector database called chroma. A vector database maps the embeddings of the schema in a multi-dimensional space and when the user enters a prompt, it gets the topmost similar tables from the schema based on the vector database. This step is taken to keep the prompt in a specific range because LLM models

have a specific prompt limit. The selected tables are passed to the LLM agent that generates an SQL query and passed the generated query to a custom tool that checks whether the SQL query is correct or not, if the SQL query is not correct then it analyzes error message and tries to regenerate the query, this step keeps repeating for a number of times till it gets the correct query. After getting the correct query it runs the query on the database and gets the data. This data is then passed to the front end and the user can view this returned data or view the SQL and generate a visualization according to the user's requirements. The visualizations can then be saved on a desired dashboard.

- Choose a Database: Users can specify the desired data source from their connected databases. When the user connects to the database it saves the schema of that database inside the vector database called chroma.
- Model Selection: Users can select the model they want to use (defog or OpenAI) for their queries.
- Communicate with the model: After selecting the model the user wants to use, they can communicate with the LLM model by providing the prompt in the chatbot interface.
- View Raw Data: After communicating with the model, the user can view the raw data fetched from the database according to the query generated by the model.

- **View SQL query:** After communicating with the model, the user can view the SQL query generated by the model.
- **Choose Visualization:** After communicating with the LLM model, the LLM model returns the SQL query, then the back-end engine returns the respected data from the SQL query, the user can select the type of visualization the users wish to see (Bar chart, line chart, pie chart, radar chart, bubble chart, scatter plot, and donut chart).
- **Save Visualization:** After generating the visualization according to the user's needs, the user can save the visualization by providing a title for the visualization and specifying the dashboard to which the visualization is to be saved.

4.3.5. Profile managment

Allows users to manage their profile by adding their avatar or changing it. Users can also change their username or password from this page.

- **Change Password/username:** From the profile page, users can edit their login information, such as username and password. This means they can change their password to something secure and customize their username so people know who they are.
- **Change Avatar:** Users can add or change their avatar image on the profile page. The avatar acts as a visual representation of the user in the system. It gives a sense of personalization.

Chapter 5

5. Results and discussion

To assess the accuracy of our chatbot interface, implemented using two models (GPT 3.5 Turbo and Defog SQLCoder), we first designed a testing framework:

1. Question
2. Create prompt
3. Generate SQL query
4. Run SQL query
5. Categorize results into correct or incorrect

First, we chose an appropriate dataset to test our chatbot interface. We chose an open-source dataset ‘Northwind Database’ which had the appropriate amount of complexity and was relevant to real-life scenarios.

We devised questions on which to test the chatbot. These questions were relevant to the selected Northwind dataset. We then curated natural-language prompts for our Large Language Models according to the questions. These prompts were tested on both GPT 3.5 Turbo and Defog. The resulting generated SQL queries for both LLMs were tested on the dataset and were categorized into ‘correct’ or ‘incorrect’ results.

The results of this test were used to optimize the performance of our chatbot which guaranteed accurate responses from the chatbot in the future. Through the results of these tests, we were also able to streamline our chatbot’s query generation process which included first generating

database schema indexing and storing them in a vector database which allows us to decrease the chatbot's inference time and also generate more accurate results. This approach also helped us to limit our maximum token size.

In conclusion, by testing both of the LLMs we not only streamlined our SQL generation technique but also were able to evaluate that both of the LLMs had their pros and cons; in general, the accuracy for 'GPT 3.5 Turbo' amounted to 60.6% and that of 'Defog SQLCoder' amounted to 64.6%.

Chapter 6

6. Conclusion and future work

This project is pioneering a new BI application designed to democratize data analysis and statistics. It focuses on user-friendliness, easy integration with existing systems, and generating powerful visualizations by leveraging NLP. This empowers users, including those with less technical expertise, to leverage data and capitalize on an untapped workforce in the data analytics. Ultimately, the project aims to revolutionize data analysis techniques and promote data-driven decision making within organizations. This technology offers organizations a lifeline, not only by introducing more data analysis capabilities but also by fostering faster and more agile business operations.

Looking ahead, the project has exciting possibilities for further development. Optimizing the AI algorithms within the NLP chatbot can potentially reduce response times and enhance user experience. Exploring deployment models beyond the current single-user installations is crucial. A Software as a Service (SaaS) model could provide scalability and cost-effectiveness for users. Alternatively, a Software as a Product model could cater to companies seeking on-premise installations. Additionally, seamless integration with existing Business Intelligence (BI) applications holds immense potential. This could leverage existing reporting/visualization infrastructure while incorporating the features and NLP capabilities of this project.

References

1. Srivastava, R., Aw, W., & Ping, W. J. (2023, February 09). Open-sourcing SQLCoder: A state-of-the-art LLM for SQL generation. <https://defog.ai/blog/open-sourcing-sqlcoder/>
2. Defog. (n.d.). sqlcoder-7b-2. Hugging Face: <https://huggingface.co/defog/sqlcoder-7b-2>
3. Guu, K., Hernandez, K., Levy, K., & Jurafsky, D. (2022, June). GraphiX-T5: Mixing Pre-Trained Transformers for Dense Language Modeling. Paper with Code: <https://paperswithcode.com/paper/graphix-t5-mixing-pre-trained-transformers>
4. Bao, Z., Dong, L., Furukawa, Y., et al. (2021, September). Counformer: Long-former Self-Attention with Conditional Routing. arXiv: <https://arxiv.org/abs/2109.05093>
5. Numbers Station. (n.d.). nsql-llama-2-7B. Hugging Face: <https://huggingface.co/NumbersStation/nsql-llama-2-7B>
6. Liu, Y., Wu, Y., Wang, H., et al. (2023, February). LLaMA: Efficient Large Language Models with Mixed-Precision Training. arXiv: <https://arxiv.org/pdf/2302.05965.pdf>
7. Vanna. (2023, January 19). AI for SQL: How Accurate Can It Be? <https://vanna.ai/blog/ai-sql-accuracy.html>

Appendix A

1. **NLP:** Natural Language Processing; an artificial intelligence technique that enables computers to interpret human language.
2. **BI:** Business Intelligence; strategies and technologies used for data analytics and data visualization that help enterprises make more data-driven decisions.
3. **SQL:** Structured Query Language; language for database manipulation.
4. **Defog SQLCoder:** Large language model to generate SQL queries.
5. **GPT 3.5:** A general purpose large language model.
6. **LLM:** Large Language Model; an Artificial Intelligence model for natural language processing.
7. **NL2SQL:** Large language model to generate SQL queries.
8. **Llama2:** A general purpose large language model.
9. **PICARD:** Large language model to generate SQL queries.
10. **RESDSQL:** Large language model to generate SQL queries.
11. **Graphix-T5:** Large language model to generate SQL queries.
12. **Tableau:** An application for business intelligence.
13. **PowerBI:** An application for business intelligence.
14. **PostgreSQL:** A database management system.

15. **MySQL:** A database management system.
16. **VRAM:** Video Random Access Memory; a dedicated memory for a computer's graphics card.
17. **GPU:** An electronic processing unit to enhance a computer's image and graphics processing.
18. **API:** Application Programming Interface; the process for two or more computer programs to communicate with each other.
19. **JavaScript:** A programming language, primarily used for web technologies.
20. **React.js:** A JavaScript library used for front-end development.
21. **Python:** A high-level, general-purpose programming language.
22. **Flask:** A Python framework used for back-end development.
23. **Chart.js:** A React.js library used for chart visualizations.
24. **XMTP:** A messaging protocol for web technologies.
25. **HTTP:** Hypertext Transfer Protocol; a protocol designed to transfer information over the internet.